FOUR-COURSE SYSTEM

COURSE 3 - DATA ENGINEERING 2026

C3-080 - Cloud & Microsoft Fabric Data Engineering

15-lesson teacher-led implementation book. Current Microsoft sources are authority for portal-sensitive behavior.

Artifact	C3-080
Status	2026 - LEARNER EDITION
Phase	C3-07 Cloud & Microsoft Fabric Data Engineering | 15 lessons | 50-70 focused hours
Prerequisite	C3-06 Gate PASS; C2B cloud foundations are recalled, while Fabric implementation is taught from first principles.
Current lane	Microsoft Fabric docs refreshed 2026-09-11 | trial 60 days | F4/F64 | up to 1 TB OneLake
Brand	Charcoal #1F2937 | Teal #14B8A6 | Soft Blue #3B82F6 | AMOLED #000

House teaching rule: mental picture -> tiny example -> guided Fabric action -> expected result -> why it worked -> break/fix -> independent changed case -> validation -> security/production reasoning -> evidence. A portal screenshot is not mastery unless the underlying data, identity, lifecycle and recovery claim is independently verified.

This book is your teacher

Use this book as the primary teacher. External videos and official documentation are visual or current-behavior companions, never hidden prerequisites.

The locked beginner-first lesson contract

01. Why it matters
02. What you learn
03. Prerequisite / starting state
04. Picture it in your head
05. New words before use
06. Useful video/source
07. Follow with me once
08. Expected result
09. Why it worked
10. Common beginner mistakes
11. If stuck / recovery
12. Small independent task
13. Validate
14. Teach it back
15. Protected review + changed retry
16. Evidence/mastery + exact next route

Rules that protect your learning

- Save a genuine attempt before opening a protected review.
- After review, close it and solve a changed retry - do not copy the reviewed answer.
- A command that runs is not mastery: validate grain, keys, counts, state, failure/recovery and evidence.
- If a new engineering concept appears, this book explains it from first principles even when Bridge skills are assumed.
- When stuck, repair only the named weak competency; do not restart the whole phase.

Contents / Index

Actual page numbers after the master-audit repair. Mentor routes use these same pages.

Module map + protected learning route

Cloud & Microsoft Fabric Data Engineering

01 Prerequisite / prior-phase Gate
02 Lesson mental model + vocabulary
03 Follow guided example once
04 Matching independent practice
05 Save genuine attempt
06 Protected review unlocks
07 Close review + changed retry
08 Validate + explain back
09 Master lesson + save evidence
10 Mini-Lab / integrated drill
11 Protected Gate A
12 If needed: protected review -> targeted repair -> changed micro-proof
13 Explicitly assign fresh Gate B/C only after repair evidence
14 Phase PASS -> next phase

What counts as mastery

- You can rebuild the competency without the lesson/review open.
- Your validation catches a plausible wrong result, not only syntax failure.
- You can state one failure mode and the recovery step.
- Your evidence names the input/output or artifact and the independent check used.

Revision rule

Revise from evidence: weak validation, weak recovery or weak explanation sends you back only to the matching lesson/repair route. Completion percentage never overrides a failed Gate.

How to use this book

For each lesson: read the mental model, inspect the current Microsoft source, follow one small guided build, save the observed result, break one assumption deliberately, repair it, then complete the changed independent task before opening review.

Fabric live-execution rule: design/local tests can prove logic and artifact quality, but they do not prove a Fabric pipeline, notebook, Warehouse/Eventhouse query, Git deployment, OneLake security path, monitoring view or capacity behavior actually ran. Those claims require observed tenant evidence.

C3-07-L01 - Cloud data-platform primitives: storage, compute, network and IAM

Focused effort: 2.5 h | Matching practice: P01 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Separate storage, compute, connectivity/control and identity responsibilities before selecting a Fabric item.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture a city utility map. Storage is the reservoir, compute is the treatment plant, network is the pipe path, and IAM is the set of valves and keys deciding who can operate each part. Fabric manages infrastructure, but you still design these boundaries.

NEW WORDS BEFORE WE USE THEM

Storage = persistent data layer. Compute = processing capacity. Network/connectivity = route between systems. IAM = identity and access management. Control plane = permission to manage items. Data plane = permission to read/write data.

USEFUL VIDEO/SOURCE

Current authority - Microsoft Fabric overview

Recommended visual - R061 Fabric overview visual

Useful segment: Topic-directed; provider page exposes no chapter timestamps.

PLAIN-ENGLISH EXPLANATION

Cloud platforms remove server ownership but not engineering ownership. OneLake supplies shared storage, Fabric capacities supply compute, workspace/item settings define control boundaries, and Microsoft Entra identities authorize people/services. Network and connection paths still matter for private/on-prem sources.

C3-07-L01 - Follow with me once

TINY WORKED EXAMPLE

Draw a boundary diagram for a CSV source -> Fabric pipeline -> Lakehouse -> Warehouse/BI consumer. Label storage, compute, control-plane and identity boundaries.

FOLLOW WITH ME ONCE

Complete architecture/L01_cloud_boundaries.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If your diagram equates workspace with storage or says SaaS removes IAM/cost/recovery duties, redraw the boundaries.

C3-07-L01 - Now prove it yourself

INDEPENDENT SMALL VERSION

Move the source behind a private/on-prem network and add the required connectivity decision.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Choose services only after workload, ownership, latency, security, recovery and cost boundaries are explicit.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P01 first. Only then open C3-082 R01. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

boundary diagram + changed network scenario. For tool lessons, real Fabric run evidence is still required before final completion. You can identify the responsibility boundary for a fresh cloud data flow.

EXACT NEXT ROUTE

Attempt P01 in C3-081 page 3. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L02 - Identity, RBAC, secrets and least privilege

Focused effort: 3.0 h | Matching practice: P02 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Design least-privilege control-plane and data-plane access without putting credentials in code or screenshots.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture an office building with reception, room keys and locked cabinets. Workspace/item roles control what someone can do in the building; OneLake data roles control which cabinets/data they may open. A secret is a key, never a label you paste on the wall.

NEW WORDS BEFORE WE USE THEM

Microsoft Entra ID = identity system. RBAC = role-based access control. Least privilege = minimum permissions needed. Secret = credential/token/key kept outside source code. Service principal = nonhuman identity for automation.

USEFUL VIDEO/SOURCE

Current authority - OneLake security overview

Recommended visual - R065 Secure and govern Fabric

Useful segment: Topic-directed; provider page exposes no chapter timestamps.

PLAIN-ENGLISH EXPLANATION

Fabric uses Microsoft Entra identities. Workspace/item roles govern control-plane actions. OneLake security roles govern data-plane access and can scope table/folder/schema plus row/column controls. Secrets belong in managed connection/identity mechanisms. Grant only what the task requires and test from the consumer identity.

C3-07-L02 - Follow with me once

TINY WORKED EXAMPLE

Compare Admin/Member/Contributor/Viewer responsibilities with a data-reader who has only appropriate OneLake Read access. Explain why configuration from an owner identity is not proof of consumer access.

FOLLOW WITH ME ONCE

Complete security/L02_identity_access_matrix.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If you solve data access by broad Contributor membership, separate item management from data-reading needs.

C3-07-L02 - Now prove it yourself

INDEPENDENT SMALL VERSION

Change a consumer from analyst to ingestion service identity and redesign permissions.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Never validate least privilege while signed in as an administrator. Test the identity that will actually consume or run the workload.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P02 first. Only then open C3-082 R02. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

access matrix + negative-access test plan. For tool lessons, real Fabric run evidence is still required before final completion. You can defend least privilege across control and data planes.

EXACT NEXT ROUTE

Attempt P02 in C3-081 page 4. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L03 - Fabric workspaces, OneLake, shortcuts and mirroring architecture

Focused effort: 2.5 h | Matching practice: P03 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Choose copy, shortcut, mirroring or native OneLake placement from ownership, freshness, transformation, security and recovery needs.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture one large warehouse campus called OneLake. Workspaces are managed zones, shortcuts are signs pointing to goods that stay elsewhere, and mirroring is a continuously refreshed copy of a supported source. Moving data is a separate decision.

NEW WORDS BEFORE WE USE THEM

OneLake = Fabric tenant-wide logical lake. Workspace = collaboration/security boundary. Shortcut = reference to data without normal copy. Mirroring = managed replication/synchronization for supported systems.

USEFUL VIDEO/SOURCE

Current authority - OneLake overview

Recommended visual - R062 Exploring OneLake

Useful segment: Topic-directed; provider page exposes no chapter timestamps.

PLAIN-ENGLISH EXPLANATION

A workspace organizes items/collaboration; OneLake is Fabric shared storage. Shortcuts reference selected external/internal data without owning a copied dataset. Mirroring adds supported database/catalog sources and can access in place or replicate depending source. Use pipelines/dataflows when movement/transformation/scheduling is the actual requirement.

C3-07-L03 - Follow with me once

TINY WORKED EXAMPLE

For four sources - same-tenant Lakehouse, S3 files, supported operational DB, daily vendor CSV - choose shortcut, mirroring, pipeline/copy or direct load and state why.

FOLLOW WITH ME ONCE

Complete architecture/L03_onelake_shortcut_mirroring.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If every source is copied, ask whether a no-copy shortcut meets freshness/governance. If every source is a shortcut, ask where transformation/ownership belongs.

C3-07-L03 - Now prove it yourself

INDEPENDENT SMALL VERSION

Change a source from static files to operational database CDC and re-evaluate.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Shortcut, mirroring and physical copy solve different ownership/freshness/transformation problems; document why you chose one.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P03 first. Only then open C3-082 R03. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

decision matrix + transfer. For tool lessons, real Fabric run evidence is still required before final completion. You can choose a fresh Fabric data-unification pattern without product-name guessing.

EXACT NEXT ROUTE

Attempt P03 in C3-081 page 5. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L04 - Fabric lakehouse and Delta tables

Focused effort: 3.0 h | Matching practice: P04 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Build/describe Files vs Tables, Bronze/Silver contracts and Delta-backed trusted tables with Spark/SQL access.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture a lakehouse with two doors: Spark engineering and SQL analytics. Files are the loading dock; Delta Tables are managed inventory with schema and transaction history. A file becoming visible does not automatically make it trusted table data.

NEW WORDS BEFORE WE USE THEM

Lakehouse = OneLake data item supporting Spark and SQL analytics. Files = file area. Table = managed Delta table. Delta = transaction-log table format. Bronze/Silver = raw/cleaned trust layers.

USEFUL VIDEO/SOURCE

Current authority - Fabric lakehouse overview

Recommended visual - Lakehouse Learn Live

Useful segment: Topic-directed; use lakehouse creation/storage/SQL portions.

PLAIN-ENGLISH EXPLANATION

A Fabric Lakehouse stores files and Delta tables in OneLake. Files are not automatically trusted tables. Table grain, keys, schema evolution, reconciliation and layer contracts remain engineering responsibilities. The same storage can be consumed by Spark and SQL analytics experiences, but each access path must be validated.

C3-07-L04 - Follow with me once

TINY WORKED EXAMPLE

Map raw orders CSV into Bronze immutable landing and Silver current-state Delta table. State key, grain, schema and reconcile counts/totals.

FOLLOW WITH ME ONCE

Complete lakehouse/L04_table_contract.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

• Choosing a Fabric item by name before stating the workload contract.

• Treating a successful portal action as proof of data correctness.

• Mixing identity, network, secret and data-grant problems together.

• Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If "uploaded successfully" is your only evidence, add row/key/schema/business-total controls.

C3-07-L04 - Now prove it yourself

INDEPENDENT SMALL VERSION

Add one corrected order and prove an idempotent Silver result.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Separate raw retention from trusted table contracts and prove table grain/schema/quality independently of the UI.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P04 first. Only then open C3-082 R04. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

contract + reconciliation. For tool lessons, real Fabric run evidence is still required before final completion. You can defend a new Lakehouse table contract and validation.

EXACT NEXT ROUTE

Attempt P04 in C3-081 page 6. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L05 - Data Factory pipelines, Copy activity and Dataflow Gen2 decision

Focused effort: 3.5 h | Matching practice: P05 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Choose Copy/pipeline, Dataflow Gen2, notebook/Spark or adjacent Copy Job from movement vs transformation vs orchestration needs.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture a logistics desk. Copy Job is the simple recurring courier, Copy activity is one movement step inside a larger pipeline, Dataflow Gen2 is a visual transformation station, and a notebook/Spark job handles code-heavy transformations.

NEW WORDS BEFORE WE USE THEM

Copy Job = simplified movement item. Copy activity = movement step inside a pipeline. Pipeline = orchestration graph. Dataflow Gen2 = Power Query-based cloud transformation item. Connector = source/destination integration.

USEFUL VIDEO/SOURCE

Current authority - Fabric Data Factory overview

Recommended visual - R063 Data Factory pipelines Learn Live

Useful segment: 03:22 objectives; 06:43 pipelines; 09:57 Copy Data; 22:48 run/monitor.

PLAIN-ENGLISH EXPLANATION

Copy activity moves data within a pipeline; pipelines orchestrate activities and parameters. Dataflow Gen2 provides Power Query transformation and, for new items since April 2026, CI/CD + Git support by default. Copy Job is a current adjacent movement/replication option; it is context, not a replacement for understanding pipelines. Tool choice follows source, transformation complexity, volume, latency and maintainability.

C3-07-L05 - Follow with me once

TINY WORKED EXAMPLE

Classify five tasks: bulk CSV landing, visual Power Query cleanup, PySpark transform, multi-step orchestration, recurring supported replication.

FOLLOW WITH ME ONCE

Complete factory/L05_tool_decision.md + factory/L05_pipeline_TODO.json. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If you choose Dataflow merely because it is visual, restate transformation complexity/scale/lifecycle needs.

C3-07-L05 - Now prove it yourself

INDEPENDENT SMALL VERSION

Change a simple copy to a schema-heavy transform and re-evaluate.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Prefer the simplest movement mechanism that satisfies the requirement; use pipelines when orchestration/control flow is actually needed.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P05 first. Only then open C3-082 R05. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

tool decision + changed case. For tool lessons, real Fabric run evidence is still required before final completion. You can justify a fresh Fabric ingestion/transformation tool.

EXACT NEXT ROUTE

Attempt P05 in C3-081 page 7. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L06 - Connections, parameters and gateway boundaries

Focused effort: 3.0 h | Matching practice: P06 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Design reusable connections/parameters and know when an on-premises gateway or delegated connection is required.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture an address book plus a secure tunnel. A connection says where/how to reach a source; parameters change non-secret run settings; a gateway bridges private/on-prem environments when required. These are different responsibilities.

NEW WORDS BEFORE WE USE THEM

Connection = reusable endpoint/auth configuration. Parameter = run-time non-secret value. Gateway = bridge to supported private/on-prem data. Credential = authentication material. Endpoint = network address.

USEFUL VIDEO/SOURCE

Current authority - Access on-premises data in Data Factory

Recommended visual - R063 Data Factory pipelines Learn Live

Useful segment: 06:43-26:41 pipeline/copy/run context; gateway specifics remain book/docs-led.

PLAIN-ENGLISH EXPLANATION

Connection identity, endpoint, authentication and network path are separate concerns. Parameters should vary environment/source/date choices without embedding secrets. Cloud sources do not automatically need an on-premises gateway; private/on-prem/network-restricted sources may. Shortcuts can also use pass-through or delegated identity depending source and design.

C3-07-L06 - Follow with me once

TINY WORKED EXAMPLE

Complete a matrix for SaaS API, public cloud storage, on-prem SQL, network-restricted S3, same-tenant OneLake shortcut.

FOLLOW WITH ME ONCE

Complete connections/L06_connection_gateway_matrix.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If every source gets a gateway, separate authentication from network reachability.

C3-07-L06 - Now prove it yourself

INDEPENDENT SMALL VERSION

Move a cloud database behind a private network and re-evaluate.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Treat network path, endpoint, identity and secret storage as separate failure domains so troubleshooting stays precise.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P06 first. Only then open C3-082 R06. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

connection matrix + transfer. For tool lessons, real Fabric run evidence is still required before final completion. You can design a fresh connection path and parameter boundary.

EXACT NEXT ROUTE

Attempt P06 in C3-081 page 8. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L07 - Fabric notebooks with PySpark

Focused effort: 3.5 h | Matching practice: P07 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Use Fabric notebook/PySpark for explicit-schema transformations and persist validated Lakehouse outputs.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture a laboratory notebook connected directly to the lakehouse. You can explore interactively, but production evidence requires explicit inputs, reusable functions, deterministic output and validation - not a cell that happened to run once.

NEW WORDS BEFORE WE USE THEM

Notebook = interactive code document. Session = attached Spark compute context. DataFrame = distributed table-like abstraction. Explicit schema = declared column types. Idempotent = safe to rerun.

USEFUL VIDEO/SOURCE

Current authority - Load lakehouse data with a notebook

Recommended visual - Spark in Fabric Learn Live

Useful segment: 12:44 prepare; 15:59 configure; 25:21 run code; 31:34 DataFrame; 37:09 transform; 1:00:31 save.

PLAIN-ENGLISH EXPLANATION

Notebooks are interactive engineering surfaces, not a substitute for reproducible code. Attach the intended Lakehouse/environment, make inputs/config explicit, transform with PySpark/SQL, write Delta tables and save row/key/business controls. Separate demonstration cells from reusable transformation functions when work will be automated.

C3-07-L07 - Follow with me once

TINY WORKED EXAMPLE

Implement the provided transformation TODO locally as syntax/static evidence, then translate it into a Fabric notebook with run evidence when available.

FOLLOW WITH ME ONCE

Complete notebooks/L07_orders_transform_TODO.py. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

• Choosing a Fabric item by name before stating the workload contract.

• Treating a successful portal action as proof of data correctness.

• Mixing identity, network, secret and data-grant problems together.

• Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If notebook success depends on hidden session state, restart/re-run from clean context and externalize parameters.

C3-07-L07 - Now prove it yourself

INDEPENDENT SMALL VERSION

Run the same logic against incremental input or a different workspace item.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Keep notebooks reproducible: explicit inputs, controlled parameters, reusable functions, deterministic output and validation cells.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P07 first. Only then open C3-082 R07. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

notebook/run output + reconciliation. For tool lessons, real Fabric run evidence is still required before final completion. You can reproduce a fresh notebook transform and defend its controls.

EXACT NEXT ROUTE

Attempt P07 in C3-081 page 9. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L08 - Spark job definitions, environments and scheduling

Focused effort: 3.5 h | Matching practice: P08 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Package a repeatable Spark job with environment/lakehouse references, parameters, scheduling and promotion caveats.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture turning a notebook experiment into a scheduled factory machine. The Spark Job Definition is the runnable job, the environment supplies runtime/dependencies, the lakehouse reference supplies data context, and scheduling decides when it runs.

NEW WORDS BEFORE WE USE THEM

Spark Job Definition = deployable Spark job item. Environment = libraries/runtime/compute settings. Default lakehouse = job data context. Promotion = move controlled definitions across environments.

USEFUL VIDEO/SOURCE

Current authority - Spark Job Definition source control

Recommended visual - Spark in Fabric Learn Live

Useful segment: Use Spark execution sections; Job Definition promotion remains docs-led.

PLAIN-ENGLISH EXPLANATION

Spark Job Definitions (SJD) are reusable batch/streaming Spark items. Fabric environments manage runtime/libraries/resources. Scheduling creates operational ownership. Git/source-control support exists but is preview-sensitive; environment/default Lakehouse references may need manual remapping across workspaces, so promotion must test them rather than assume IDs travel safely.

C3-07-L08 - Follow with me once

TINY WORKED EXAMPLE

Turn L07 transformation into a job-spec package: main file, parameters, environment/lakehouse mapping, schedule, expected evidence and failure recovery.

FOLLOW WITH ME ONCE

Complete spark_jobs/L08_job_definition_TODO.py + spark_jobs/L08_promotion_contract.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If deployment assumes source workspace IDs work everywhere, add explicit remapping/validation.

C3-07-L08 - Now prove it yourself

INDEPENDENT SMALL VERSION

Promote Dev -> Test with a different Lakehouse/environment name.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Promotion is incomplete until environment/lakehouse references are remapped and the deployed job is smoke-tested with production-safe inputs.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P08 first. Only then open C3-082 R08. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

job definition + schedule + promotion check. For tool lessons, real Fabric run evidence is still required before final completion. You can package and promote a fresh Spark job safely.

EXACT NEXT ROUTE

Attempt P08 in C3-081 page 10. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L09 - Fabric Warehouse and T-SQL serving layer

Focused effort: 3.0 h | Matching practice: P09 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Choose Warehouse vs Lakehouse SQL serving and define stable relational serving objects without duplicating raw engineering data.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture a curated service counter over OneLake. The Warehouse is SQL-first serving with schemas, tables, views and transactional capabilities. Its job is not to replace the lakehouse; it gives relational consumers a governed contract.

NEW WORDS BEFORE WE USE THEM

Warehouse = SQL-first analytical database item. T-SQL = SQL dialect used by Microsoft SQL engines. View = stored query interface. Schema = namespace/security/ownership grouping. Transaction = atomic multi-step change where supported.

USEFUL VIDEO/SOURCE

Current authority - Fabric Data Warehouse overview

Recommended visual - Warehouse Learn Live

Useful segment: 16:13 fundamentals; 41:45 Fabric warehouse; 49:54 query/transform; 1:16:08 secure/monitor.

PLAIN-ENGLISH EXPLANATION

Fabric Warehouse is SQL-first serving/warehousing on OneLake. A Lakehouse SQL endpoint and Warehouse solve overlapping but different modeling/serving needs. Curated dimensions/facts/views need explicit grain and reconciliation. Schema changes should be source-controlled where supported.

C3-07-L09 - Follow with me once

TINY WORKED EXAMPLE

Design customer/order serving schema and daily-store-revenue view from Silver orders.

FOLLOW WITH ME ONCE

Complete warehouse/L09_schema_TODO.sql + warehouse/L09_serving_views_TODO.sql. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If you copy raw data just to get SQL access, ask whether Lakehouse SQL endpoint is enough.

C3-07-L09 - Now prove it yourself

INDEPENDENT SMALL VERSION

Add a second serving mart requiring relational constraints/schema ownership and revisit Warehouse choice.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Serving models require stable grain, ownership, permissions and reconciliation, not just a successful SELECT.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P09 first. Only then open C3-082 R09. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

SQL objects + grain + reconciliation. For tool lessons, real Fabric run evidence is still required before final completion. You can defend a fresh serving-layer choice and SQL contract.

EXACT NEXT ROUTE

Attempt P09 in C3-081 page 11. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L10 - Eventstreams, Eventhouse and KQL foundations

Focused effort: 3.5 h | Matching practice: P10 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Design Eventstream -> Eventhouse/KQL path and query operational event data with KQL while distinguishing it from batch serving.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture a live control room. Eventstream routes incoming signals, Eventhouse stores/serves time-series event data, and KQL lets you ask time-oriented operational questions quickly. This is a different serving pattern from a dimensional warehouse.

NEW WORDS BEFORE WE USE THEM

Eventstream = real-time ingestion/routing item. Eventhouse = store optimized for event/time-series analytics. KQL = Kusto Query Language. Bin = bucket timestamps into windows. Activator = action/alert capability in Real-Time Intelligence.

USEFUL VIDEO/SOURCE

Current authority - Real-Time Intelligence documentation

Recommended visual - Fabric overview visual

Useful segment: Topic-directed; RTI implementation is docs/book-led.

PLAIN-ENGLISH EXPLANATION

Real-Time Intelligence uses Eventstream to ingest/transform/route real-time events and Eventhouse/KQL databases for scalable event/time-series analytics. KQL filters, summarizes and bins event time naturally. Warehouse remains appropriate for relational serving; Eventhouse is not "faster warehouse."

C3-07-L10 - Follow with me once

TINY WORKED EXAMPLE

Write KQL for site-level 5-minute event count/average temperature and suspect events, and draw Eventstream routing.

FOLLOW WITH ME ONCE

Complete rti/L10_telemetry_queries_TODO.kql + rti/L10_eventstream_design.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If you put all historical relational serving into Eventhouse, restate query pattern, latency and model needs.

C3-07-L10 - Now prove it yourself

INDEPENDENT SMALL VERSION

Add a near-real-time alert/Activator need and revise the path.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Choose Eventhouse/KQL for event/time-series access patterns; do not move a relational serving workload there merely because it is real-time branded.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P10 first. Only then open C3-082 R10. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

KQL + event path + validation. For tool lessons, real Fabric run evidence is still required before final completion. You can design/query a fresh event analytics path.

EXACT NEXT ROUTE

Attempt P10 in C3-081 page 12. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L11 - Incremental and CDC-style implementation patterns

Focused effort: 3.0 h | Matching practice: P11 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Apply stable-key/watermark/MERGE/reconciliation logic and choose mirroring/CDC when custom polling is unnecessary.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture a bookmark that says "processed through this change." Incremental loading advances that state only after a safe publish. CDC carries inserts/updates/deletes; mirroring can provide managed replication for supported sources. Every rerun must converge.

NEW WORDS BEFORE WE USE THEM

Watermark = durable high-water mark for incremental progress. CDC = change data capture. MERGE/upsert = deterministic insert/update pattern. Tombstone/delete event = representation of deletion. Replay = rerun historical changes safely.

USEFUL VIDEO/SOURCE

Current authority - Create a Copy Job

Recommended visual - R063 Data Factory pipelines Learn Live

Useful segment: 09:57-26:41 movement/run-monitor concepts; CDC/Copy Job specifics are current docs-led.

PLAIN-ENGLISH EXPLANATION

Incremental correctness is independent of portal success. Persist a durable watermark, read only eligible changes, merge by a stable key, reconcile inserted/updated counts and prove rerun idempotence. Mirroring can replace custom polling for supported sources; shortcuts are references, not change-processing engines.

C3-07-L11 - Follow with me once

TINY WORKED EXAMPLE

Use the local reference harness on orders_initial + orders_incremental. Prove one update and one insert, then rerun and prove no duplicate keys.

FOLLOW WITH ME ONCE

Complete incremental/L11_incremental_contract.md + scripts/reference_controls.py. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

• Choosing a Fabric item by name before stating the workload contract.

• Treating a successful portal action as proof of data correctness.

• Mixing identity, network, secret and data-grant problems together.

• Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If you move the watermark before validating the target, repair commit ordering.

C3-07-L11 - Now prove it yourself

INDEPENDENT SMALL VERSION

Add a late correction whose updated_at is newer but business date older.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Advance state only after safe publish; preserve replay/backfill evidence; a duplicate-free result must be proven under rerun.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P11 first. Only then open C3-082 R11. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

merge controls + rerun proof. For tool lessons, real Fabric run evidence is still required before final completion. You can implement a fresh incremental current-state load safely.

EXACT NEXT ROUTE

Attempt P11 in C3-081 page 13. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L12 - Git integration, database projects and deployment pipelines

Focused effort: 3.0 h | Matching practice: P12 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Use branch/PR/source-control and deployment pipelines while respecting item-specific lifecycle limitations and environment references.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture source control as blueprints, not the building contents. Git/deployment can version item definitions and metadata, while OneLake data remains separate. Promotion therefore needs both code/definition control and data-state validation/remapping.

NEW WORDS BEFORE WE USE THEM

Git integration = workspace item definitions synchronized with a repository. Deployment pipeline = promotion across Fabric stages. Database project = versionable warehouse schema project. Remap = update environment-specific references after promotion.

USEFUL VIDEO/SOURCE

Current authority - Lakehouse Git & deployment pipelines

Recommended visual - R064 Fabric items in Git

Useful segment: Topic-directed; provider page exposes no chapter timestamps.

PLAIN-ENGLISH EXPLANATION

Fabric Git/deployment support is item-specific and evolving. Warehouse Git integration/database-project workflow is preview in the current docs; Lakehouse Git tracks metadata, not data files/tables. SJD source control is also preview-sensitive and workspace references may need remapping. Use feature branches, review, explicit config mapping and post-deploy validation.

C3-07-L12 - Follow with me once

TINY WORKED EXAMPLE

Create Dev -> Test release plan for notebook, pipeline/dataflow, Spark job, Lakehouse metadata and Warehouse schema. Mark what is committed vs separately provisioned.

FOLLOW WITH ME ONCE

Complete cicd/L12_release_plan.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

• Choosing a Fabric item by name before stating the workload contract.

• Treating a successful portal action as proof of data correctness.

• Mixing identity, network, secret and data-grant problems together.

• Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If your plan says "Git backs up the data," separate item definition/metadata from OneLake table/file data.

C3-07-L12 - Now prove it yourself

INDEPENDENT SMALL VERSION

Introduce a Test workspace with different connection/environment IDs and revise promotion.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Version definitions, not data. A Git-green release still needs data-state, dependency and post-deploy validation.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P12 first. Only then open C3-082 R12. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

commit map + deployment checklist + rollback. For tool lessons, real Fabric run evidence is still required before final completion. You can defend a fresh Fabric change from branch to validated target.

EXACT NEXT ROUTE

Attempt P12 in C3-081 page 14. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L13 - Monitoring and performance optimization

Focused effort: 3.5 h | Matching practice: P13 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Diagnose pipeline/Spark/Warehouse/Eventhouse/capacity issues from run/metric evidence before changing settings.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture a hospital monitor. A green light alone is weak. You need run history, duration, throughput, error detail, capacity pressure and business reconciliation, plus a recovery action linked to each alert.

NEW WORDS BEFORE WE USE THEM

Monitoring Hub = centralized activity/run monitoring. Capacity Metrics app = capacity consumption/pressure visibility. SLI = measured reliability signal. SLO = target for the signal. Runbook = detect/diagnose/recover/validate instructions.

USEFUL VIDEO/SOURCE

Current authority - Fabric Monitoring Hub

Recommended visual - Warehouse Learn Live

Useful segment: 1:16:08 onward for secure/monitor orientation; Monitoring Hub docs control exact behavior.

PLAIN-ENGLISH EXPLANATION

Monitor item run status, duration, throughput, errors, Spark job/stage evidence, Warehouse query behavior, Eventhouse ingestion/query logs and capacity utilization. Eventstream workspace monitoring is currently preview and exposes KQL-queryable status/metrics/error tables when enabled. Tune only after correctness and bottleneck evidence.

C3-07-L13 - Follow with me once

TINY WORKED EXAMPLE

Diagnose three supplied incidents: slow copy, Spark failure, Eventstream error spike. Complete signals -> hypothesis -> action -> validation.

FOLLOW WITH ME ONCE

Complete monitoring/L13_monitoring_matrix.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If the first response to slowness is "increase capacity," prove whether source/sink/query design or concurrency is the actual bottleneck.

C3-07-L13 - Now prove it yourself

INDEPENDENT SMALL VERSION

Change an incident from compute saturation to source throttling and revise response.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Every alert needs an owner and next action; operational metrics without recovery decisions become dashboard decoration.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P13 first. Only then open C3-082 R13. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

incident matrix + validated action. For tool lessons, real Fabric run evidence is still required before final completion. You can diagnose a fresh Fabric incident from evidence.

EXACT NEXT ROUTE

Attempt P13 in C3-081 page 15. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L14 - Fabric security & governance: access scopes, masking, labels, audit and OneLake security

Focused effort: 3.0 h | Matching practice: P14 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Apply and test least-privilege governance using workspace/item control plane plus OneLake/SQL data-plane controls, masking/labels/audit.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture layered doors. Workspace/item permissions are control plane; OneLake roles are data plane. Because access is grant-based and may come from multiple paths, security testing must use the real consumer identity and prove both allowed and blocked access.

NEW WORDS BEFORE WE USE THEM

OneLake security role = data-plane grant. RLS = row-level security. CLS = column-level security. Masking = display protection, not full authorization. Sensitivity label = governance classification. Audit = record of access/activity.

USEFUL VIDEO/SOURCE

Current authority - OneLake security roles

Recommended visual - R065 Secure and govern Fabric

Useful segment: Topic-directed; provider page exposes no chapter timestamps.

PLAIN-ENGLISH EXPLANATION

Current OneLake security separates control-plane permissions from data-plane roles. OneLake roles grant Read/ReadWrite to selected tables/folders/schemas and can refine access with RLS/CLS. They are grant-based: another permission path can still grant access. Default/broad reader roles must be reviewed. Dynamic masking is not a substitute for authorization. Audit and sensitivity metadata support governance evidence.

C3-07-L14 - Follow with me once

TINY WORKED EXAMPLE

Design Store analyst, Finance analyst, pipeline service and platform admin access. Include expected allowed/blocked actions and a consumer-identity test.

FOLLOW WITH ME ONCE

Complete security/L14_security_matrix.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

- Choosing a Fabric item by name before stating the workload contract.
- Treating a successful portal action as proof of data correctness.
- Mixing identity, network, secret and data-grant problems together.
- Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If you test only as workspace Admin, your access proof is invalid. Use a consumer identity/role plan and negative checks.

C3-07-L14 - Now prove it yourself

INDEPENDENT SMALL VERSION

Add a contractor who may read East-region orders without customer email.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

OneLake is grant-based; another permission path can still grant access. Security proof therefore includes identity-specific negative tests.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P14 first. Only then open C3-082 R14. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

role matrix + negative test + audit evidence. For tool lessons, real Fabric run evidence is still required before final completion. You can design and test a fresh OneLake/SQL access policy.

EXACT NEXT ROUTE

Attempt P14 in C3-081 page 16. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

C3-07-L15 - Capacity/cost awareness and DP-700 objective mapping

Focused effort: 2.5 h | Matching practice: P15 | C3-06 Gate PASS assumed

WHY IT MATTERS

Fabric adds a managed platform, but correctness, security, reproducibility and operational ownership remain engineering responsibilities.

WHAT YOU LEARN

Operate within capacity/cost constraints and map genuine C3-07 evidence to the current DP-700 domains without turning the phase into exam cramming.

PREREQUISITE

C3-06 is mastered. C2B cloud fundamentals may be recalled, but this Fabric-specific behavior is taught explicitly. No hidden portal knowledge is assumed.

PICTURE IT IN YOUR HEAD

Picture one electricity meter powering many Fabric workloads. Capacity Units are shared compute pressure. Scheduling, query shape, Spark jobs and copy workloads compete for capacity, so cost/performance evidence is an engineering responsibility, not finance-only.

NEW WORDS BEFORE WE USE THEM

Capacity Unit (CU) = Fabric compute consumption unit. Throttling = workload delayed/rejected due to capacity pressure. Trial capacity = temporary Fabric capacity. DP-700 = Fabric Data Engineer certification exam; objective map is evidence organization, not mastery by itself.

USEFUL VIDEO/SOURCE

Current authority - DP-700 study guide

Recommended visual - R061 Fabric overview visual

Useful segment: Topic-directed; certification objective wording is refreshed from official study guide.

PLAIN-ENGLISH EXPLANATION

Fabric capacity is shared compute. Monitor CU pressure/contention, avoid stacking heavy Spark/Eventhouse/Copy jobs without need, and schedule deliberately. Trial capacity is time-limited, so preserve definitions/evidence and prioritize hands-on execution. DP-700 skills measured as of 21 July 2026 are three balanced domains (30-35% each): implement/manage, ingest/transform, monitor/optimize.

C3-07-L15 - Follow with me once

TINY WORKED EXAMPLE

Create capacity-aware operating schedule plus evidence map from C3-07 artifacts to the three current DP-700 domains.

FOLLOW WITH ME ONCE

Complete capacity/L15_capacity_dp700_map.md. Save your own artifact/evidence; do not copy a finished solution.

EXPECTED RESULT

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

WHY IT WORKED

The design separates platform responsibilities, makes identity/data boundaries explicit, and requires an independent correctness check rather than trusting the portal surface.

BREAK / FIX / DEBUG

Change one boundary deliberately: item/tool choice, identity, source path, parameter, schema, incremental state, environment reference, security grant, monitoring signal or capacity constraint. Predict what should fail, capture the observed evidence, then repair only the failing boundary.

COMMON BEGINNER MISTAKES

• Choosing a Fabric item by name before stating the workload contract.

• Treating a successful portal action as proof of data correctness.

• Mixing identity, network, secret and data-grant problems together.

• Skipping rerun/recovery or negative security evidence.

RECOVERY IF STUCK

If your DP-700 map lists topics you have never implemented/evidenced, mark them as study gaps, not mastery.

C3-07-L15 - Now prove it yourself

INDEPENDENT SMALL VERSION

Cut available capacity in half and reschedule workloads while preserving SLA.

VALIDATION

Use at least one independent control: source-to-target counts/keys/totals, rerun idempotence, query/KQL control, expected blocked security test, run/monitoring evidence, or post-deploy smoke check as appropriate.

PRODUCTION / ARCHITECTURE REASONING

Capacity is a shared systems constraint. Optimize by workload shape, concurrency and schedule using observed CU/performance evidence.

TEACH IT BACK

Explain the design in your own words for 60-90 seconds: what owns data, what computes, which identity acts, how correctness is proved, how failure is recovered and how a change is promoted safely.

PROTECTED REVIEW + CHANGED RETRY

Save P15 first. Only then open C3-082 R15. Close review and solve the changed retry without reopening it. Preserve the failed attempt; append the retry.

EVIDENCE / MASTERY

capacity schedule + DP700 evidence map. For tool lessons, real Fabric run evidence is still required before final completion. You can defend a fresh cost/capacity plan and evidence-based certification map.

EXACT NEXT ROUTE

Attempt P15 in C3-081 page 17. If evidence passes, continue. If weak, repair only this competency and complete a fresh changed retry.

Mastery checklist + revision quick reference

- [ ] Every lesson: guided rebuild completed, genuine independent attempt saved, validation recorded.
- [ ] Protected review opened only after the attempt; changed retry solved after closing the review.
- [ ] Explain-back: business/engineering purpose, grain or state, one failure mode, one recovery step, one validation check.
- [ ] Mini-Lab / integrated drill completed with reproducible evidence.
- [ ] Gate A attempted independently. If NEEDS_REPAIR: review viewed, targeted repair completed, changed micro-proof recorded, then a fresh B/C assigned explicitly.
- [ ] No unresolved critical failure, secret/private data leakage, fabricated live-run evidence or copied Gate solution.
- [ ] Evidence is saved with enough context for another person to reproduce or inspect it.

Revision quick reference

Wrong result / wrong grain	Return to validation + data contract / grain lesson; reproduce smallest failing case.
Works once, rerun breaks	Return to idempotence/state/checkpoint/recovery lesson; prove clean replay.
Cannot explain failure	Return to mental model + break/fix section; write the failure boundary in your own words.
Gate NEEDS_REPAIR	Open only its protected review, repair named weak competency, solve changed micro-proof, then assign fresh retest.
Tool/vendor changed	Use the current official source for syntax/UI, but keep the book's durable engineering contract and validation.